

ARTIFICIAL INTELLIGENCE – ASSESSMENT TOOL 2

REPORT

QUESTION 1

Topic

*Emergency Medicine Delivery using Greedy Best-First Search and A Search**

Problem Statement

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic, where roads may suddenly become inaccessible due to flooding and weather conditions influence travel cost. The drone has access to heuristic estimates based on straight-line distance, receives partial real-time updates regarding blocked paths, and experiences variable movement costs depending on terrain and weather. Because emergency response is critical, the drone must rapidly determine the safest and shortest route while minimizing delivery delay and operational risk.

Problem Understanding

Emergency medicine delivery during natural disasters is a time-critical application of Artificial Intelligence. In flood-affected regions, roads can become blocked unexpectedly, weather conditions can increase travel time, and the environment changes continuously. Traditional path-planning methods may not respond efficiently to these dynamic conditions.

The AI-powered drone acts as an intelligent agent that observes the environment through sensors and GPS data. It must determine the most suitable route to deliver medicines while minimizing travel time, energy consumption, and risk.

The search problem consists of:

- **Initial State:** Drone located at the emergency response center.
- **Goal State:** Deliver medicines safely to the affected location.
- **State Space:** All possible routes from the starting point to the destination.
- **Actions:** Move from one connected location to another.

- **Path Cost:** Travel cost based on distance, weather, terrain, and blocked roads.
- **Heuristic Function:** Estimated straight-line distance from the current location to the destination.

Since emergency response requires quick decision-making, heuristic search algorithms such as Greedy Best-First Search and A* Search are suitable candidates for route planning.

Method Selection

Two informed search algorithms are selected for solving this problem:

1. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the node that appears closest to the goal according to the heuristic function $h(n)$. It ignores the cost already travelled and always expands the node with the smallest estimated distance to the destination.

Evaluation Function:

$$f(n) = h(n)$$

where:

- $h(n)$ = estimated distance from node n to the goal.

This algorithm is extremely fast because it always chooses the most promising direction.

2. A* Search

A* Search combines both the actual path cost and heuristic estimate.

Evaluation Function:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = actual travel cost from the starting point.
- $h(n)$ = estimated remaining distance to the destination.

Unlike Greedy Search, A* considers both efficiency and optimality, making it more suitable for real-world navigation.

Solution

(i) Explain how Greedy Best-First Search behaves in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search always chooses the next location that appears closest to the destination according to the heuristic value.

For example:

Drone Base



Hospital

Suppose

- $h(A)=8$
- $h(B)=5$
- $h(C)=2$

Greedy Search immediately selects node **C** because it has the smallest heuristic value.

Strengths

- Very fast decision making.
- Requires fewer node expansions.
- Suitable when immediate response is required.
- Low memory consumption.

Weaknesses

- Does not consider actual travel cost.
- May choose flooded or blocked roads.
- Does not always produce the shortest or safest path.
- Performance decreases in highly dynamic environments.

(ii) Explain how A* handles the same situation.

A* evaluates every possible path using both actual travel cost and heuristic estimate.

Example:

Route	$g(n)$	$h(n)$	$f(n)$
-------	--------	--------	--------

Base $\rightarrow$ A	4	7	11
----------------------	---	---	----

Base $\rightarrow$ B	6	3	9
----------------------	---	---	---

Although Route B has a higher travel cost initially, its total estimated cost is lower.

Therefore, A* selects Route B because

$$f(B)=9 < f(A)=11$$

A* balances speed and accuracy by considering both the distance already travelled and the estimated remaining distance.

(iii) Impact of Heuristic Accuracy

The heuristic function greatly influences the performance of both algorithms.

Accurate Heuristic

- Faster search
- Fewer expanded nodes
- Better path selection
- Reduced computation

Inaccurate Heuristic

- Wrong route selection
- More unnecessary exploration
- Increased execution time
- Reduced efficiency

Greedy Search is highly dependent on heuristic accuracy because it relies only on $h(n)$.

A* is more robust because it also considers $g(n)$.

(iv) Effect of Dynamic Changes

Floods may suddenly block roads.

When this happens:

- Previously selected paths become invalid.
- The search algorithm must recompute a new path.
- Greedy Search may continue following misleading heuristic estimates.
- A* recalculates the total path cost and usually identifies a safer alternative.

Real-time sensor updates help the drone avoid dangerous routes and continue the mission successfully.

(v) Recommended Algorithm

The recommended algorithm is **A* Search**.

Justification

- Produces optimal routes.
- Considers both travel cost and estimated distance.
- Handles changing environments better.
- Avoids unnecessarily expensive routes.

- Suitable for emergency medicine delivery where safety and reliability are essential.

Although Greedy Search is faster, it may choose unsafe or blocked paths because it ignores actual travel costs.

Algorithm

*Algorithm: A Search for Emergency Medicine Delivery**

1. Initialize the open list with the starting node.
2. Calculate $g(n)$, $h(n)$, and $f(n)$ for the start node.
3. Remove the node with the smallest $f(n)$.
4. If the goal is reached, terminate the search.
5. Generate all neighbouring nodes.
6. Update their $g(n)$, $h(n)$, and $f(n)$ values.
7. Add valid neighbours to the open list.
8. Repeat until the destination is reached.
9. Display the optimal route and total travel cost.

Advantages

- Finds optimal solutions.
- Efficient path planning.
- Suitable for real-time navigation.
- Handles dynamic environments.
- Reduces delivery delay.

Applications

- Emergency medicine delivery
- Drone navigation

- GPS navigation systems
- Autonomous vehicles
- Disaster management
- Robotics

Result

The A* Search algorithm successfully identifies the safest and minimum-cost route for delivering emergency medicines in a flood-affected region. By combining the actual travel cost with the heuristic estimate, it provides an optimal balance between speed, safety, and reliability. Compared to Greedy Best-First Search, A* is better suited for dynamic environments where road conditions and travel costs change frequently.

QUESTION 2

Smart Traffic Signal Optimization using Hill Climbing and Simulated Annealing / Genetic Algorithm

Problem Statement

A smart city implements an Artificial Intelligence system to optimize traffic signal timings across hundreds of intersections. The primary objectives are to minimize total waiting time, reduce fuel consumption, and decrease traffic congestion. The system continuously adjusts traffic signal timings according to changing traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the optimization process may become trapped in locally optimal solutions. Therefore, advanced optimization techniques are required to obtain efficient traffic signal schedules.

Problem Understanding

Traffic congestion is one of the major challenges in modern smart cities. Poor traffic signal management increases vehicle waiting time, fuel consumption, air pollution, and travel delays. Since traffic volume changes continuously throughout the day, manually controlling traffic signals is inefficient.

Artificial Intelligence can solve this optimization problem by continuously analyzing real-time traffic data collected from sensors, cameras, and GPS devices. The AI system must determine the optimal signal timing for each intersection while satisfying multiple objectives.

The optimization problem consists of:

- **Initial State:** Current traffic signal timings at all intersections.
- **Goal State:** Traffic signal configuration that minimizes waiting time, fuel consumption, and congestion.
- **State Space:** All possible combinations of signal timings for every intersection.
- **Operators:** Increase or decrease green signal duration, adjust red signal timing, or change traffic phases.
- **Objective Function:** Minimize total waiting time, fuel consumption, and traffic congestion.

Since the number of possible signal combinations is extremely large, traditional search algorithms become computationally expensive. Optimization algorithms such as Hill Climbing, Simulated Annealing, and Genetic Algorithms provide efficient solutions.

Method Selection

To solve this optimization problem, the following Artificial Intelligence techniques are considered:

1. Hill Climbing

Hill Climbing is a local search optimization algorithm that repeatedly improves the current solution by moving toward a neighboring solution with a better evaluation value. It continues until no better neighboring solution exists.

Advantages

- Simple to implement.
- Fast execution.
- Requires less memory.

- Suitable for small optimization problems.

Limitations

- Can become trapped in local maxima.
- Cannot escape plateaus.
- May fail to find the global optimum.

2. Simulated Annealing

Simulated Annealing improves upon Hill Climbing by occasionally accepting worse solutions during the early stages of the search. This allows the algorithm to escape local maxima and continue searching for better solutions.

Advantages

- Escapes local maxima.
- Finds better global solutions.
- Suitable for large optimization problems.
- Handles dynamic search spaces effectively.

3. Genetic Algorithm

Genetic Algorithm is an evolutionary optimization technique inspired by natural selection. It maintains a population of candidate solutions and improves them through selection, crossover, and mutation.

Advantages

- Explores multiple solutions simultaneously.
- Avoids premature convergence.
- Suitable for complex optimization problems.
- Produces high-quality solutions for large search spaces.

Solution

(i) Formulate the problem as an optimization problem and explain why traditional search is inefficient.

The traffic signal optimization problem can be formulated as an optimization problem where the objective is to determine the best signal timings that minimize:

- Total vehicle waiting time
- Fuel consumption
- Traffic congestion

Objective Function

Minimize:

- Total Waiting Time
- Fuel Consumption
- Congestion Index

Subject to:

- Minimum green signal duration.
- Maximum red signal duration.
- Pedestrian crossing requirements.
- Emergency vehicle priority.

Traditional search algorithms such as Breadth-First Search or Depth-First Search become inefficient because the search space grows exponentially as the number of intersections increases. Exploring every possible traffic signal configuration would require excessive computation time and memory.

(ii) Explain how Hill Climbing works in this scenario and identify its limitations.

Hill Climbing begins with the current traffic signal schedule and evaluates neighboring signal configurations.

For example:

Current green time:

Intersection A = 30 seconds

Neighboring solutions:

28 seconds

29 seconds

31 seconds

32 seconds

The algorithm selects the configuration that produces the lowest waiting time.

This process continues until no neighboring configuration provides further improvement.

Limitations

- Stops at the first local optimum.
- Cannot escape local maxima.
- Performs poorly on flat search spaces (plateaus).
- Sensitive to the initial solution.

(iii) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

Local Maxima

A traffic signal schedule appears to be optimal within a small region but is not the best overall solution.

Example:

A particular intersection experiences reduced waiting time, while nearby intersections become heavily congested.

Plateaus

Many neighboring solutions have identical evaluation values.

Example:

Changing the green signal duration by one second does not significantly affect traffic flow.

The algorithm cannot determine which direction to move.

Ridges

The optimal solution lies along a narrow path that requires multiple coordinated changes.

Example:

Improving traffic flow requires simultaneous adjustment of several connected intersections rather than a single traffic signal.

Hill Climbing often struggles to follow such paths.

(iv) Explain how Simulated Annealing or Genetic Algorithms overcome these limitations.

Simulated Annealing

Simulated Annealing introduces randomness into the search process.

During the early stages, it may temporarily accept worse solutions with a certain probability.

This allows the algorithm to:

- Escape local maxima.
- Explore new regions of the search space.
- Discover better global solutions.

As the temperature decreases, the algorithm becomes more selective and gradually converges to an optimal solution.

Genetic Algorithm

Genetic Algorithm maintains a population of candidate traffic signal schedules.

The algorithm performs:

- **Selection:** Choose the best-performing schedules.
- **Crossover:** Combine two good schedules.
- **Mutation:** Introduce small random changes.

This evolutionary process enables continuous improvement while avoiding premature convergence.

(v) Recommend the best approach and justify your choice based on scalability and adaptability.

The recommended approach is the **Genetic Algorithm**.

Justification

- Handles very large search spaces efficiently.
- Optimizes multiple objectives simultaneously.
- Easily adapts to changing traffic conditions.
- Explores many possible solutions at once.
- Produces near-optimal traffic signal schedules.

For real-time smart city applications involving hundreds of intersections, the Genetic Algorithm provides better scalability, flexibility, and robustness than Hill Climbing. Simulated Annealing is also an effective alternative, particularly when computational resources are limited.

Algorithm

Algorithm: Genetic Algorithm for Smart Traffic Signal Optimization

1. Initialize a population of random traffic signal schedules.
2. Evaluate the fitness of each schedule based on waiting time, fuel consumption, and congestion.
3. Select the best-performing schedules.
4. Apply crossover to generate new schedules.

5. Apply mutation to introduce diversity.
6. Replace weaker schedules with newly generated schedules.
7. Repeat the process until the stopping condition is satisfied.
8. Display the optimal traffic signal schedule.

Advantages

- Finds near-optimal solutions.
- Efficient for large optimization problems.
- Supports multiple optimization objectives.
- Adapts to changing traffic conditions.
- Reduces congestion and travel time.
- Improves fuel efficiency.
- Suitable for real-time smart city applications.

Applications

- Smart city traffic management.
- Adaptive traffic signal control.
- Intelligent transportation systems.
- Urban road network optimization.
- Emergency vehicle routing.
- Public transportation scheduling.
- Smart infrastructure management.

Result

The Smart Traffic Signal Optimization system successfully improves traffic flow by minimizing waiting time, fuel consumption, and congestion using Artificial Intelligence optimization techniques. Hill Climbing provides a simple

local optimization method but is limited by local maxima and plateaus. Simulated Annealing improves exploration by escaping local optima, while the Genetic Algorithm offers the most effective solution by searching multiple candidate solutions simultaneously. Therefore, the Genetic Algorithm is the most suitable approach for large-scale, dynamic smart city traffic management systems due to its scalability, adaptability, and ability to produce high-quality optimization results.

QUESTION 3

Autonomous Mars Rover using Online Search Agent

Problem Statement

An autonomous rover is deployed on Mars to explore unknown terrain. The rover must navigate safely to collect scientific samples, avoid hazards such as rocks and craters, operate with limited sensing capability, and make decisions without having a complete map of its surroundings. As the rover explores, it continuously updates its knowledge about the environment, making this a real-time decision-making problem that requires an Online Search Agent.

Problem Understanding

Mars exploration is one of the most challenging applications of Artificial Intelligence. Unlike robots operating in known environments, a Mars rover cannot rely on a complete map because the terrain is unknown and continuously changing. Communication delays between Earth and Mars make remote control impractical, requiring the rover to make autonomous decisions.

The rover uses sensors, cameras, laser scanners, and navigation systems to perceive its surroundings. It gradually builds knowledge of the environment while searching for safe paths to reach scientific targets. Since new obstacles can appear unexpectedly, the rover must adapt its decisions in real time.

The search problem consists of:

- **Initial State:** Rover positioned at the landing site.
- **Goal State:** Reach the target location safely and collect scientific samples.

- **State Space:** All possible locations that the rover may explore.
- **Actions:** Move forward, move backward, turn left, turn right, avoid obstacles, collect samples, and update the map.
- **Environment:** Unknown, partially observable, dynamic, and sequential.

Because the rover discovers the environment during execution, an **Online Search Agent** is more suitable than traditional offline search algorithms.

Method Selection

The Mars rover problem is solved using an **Online Search Agent**.

Unlike offline search algorithms, an online search agent does not require complete knowledge of the environment before beginning the search. Instead, it explores the environment step by step, updates its internal model based on newly observed information, and continuously replans its actions.

Characteristics of an Online Search Agent

- Operates in unknown environments.
- Makes decisions while exploring.
- Continuously updates knowledge.
- Adapts to dynamic changes.
- Suitable for autonomous robotic systems.

Solution

(i) Explain why this problem requires an Online Search Agent instead of Offline Search.

Offline search algorithms assume that the entire environment is known before the search begins. They generate a complete path from the start to the goal before execution.

In contrast, the Mars rover operates in an unknown environment where:

- Terrain information is incomplete.

- New obstacles are discovered during exploration.
- Hazards such as rocks and craters cannot be predicted.
- Sensor readings continuously change.

Therefore, the rover cannot compute a complete path in advance.

An Online Search Agent is required because it:

- Explores the environment while moving.
- Updates knowledge after every action.
- Replans its path whenever new obstacles are detected.
- Makes autonomous decisions without waiting for instructions from Earth.

(ii) Analyze the challenges of partial observability and dynamic environments.

Partial Observability

The rover cannot observe the entire Martian surface at once.

It only receives information from nearby sensors.

Examples include:

- Hidden rocks behind hills.
- Unknown craters.
- Dust storms reducing visibility.
- Limited camera range.

As a result, the rover must make decisions with incomplete information.

Dynamic Environment

The Martian environment changes continuously.

Examples include:

- Dust storms covering safe paths.

- Loose rocks blocking movement.
- Battery level decreasing during exploration.
- Communication delays with Earth.

The rover must adapt its route whenever these changes occur.

(iii) Describe how the rover builds and updates its internal model.

The rover maintains an **internal map** of the explored environment.

The process includes:

1. Sense the surroundings using cameras and sensors.
2. Identify safe and hazardous regions.
3. Store newly discovered information.
4. Update the internal map.
5. Recalculate the best path.
6. Continue exploration.

For example:

Initially:

Landing Site → Unknown Area

After exploration:

Landing Site

|

Safe Path

|

Rock (Obstacle)

|

Alternative Path

|

Sample Location

The internal model becomes more accurate as exploration continues, allowing safer navigation.

(iv) Compare Online Search with Uninformed and Informed Search Strategies.

Feature	Online Search	Uninformed Search	Informed Search
Environment Knowledge	Unknown	Known	Known
Uses Heuristic	Optional	No	Yes
Real-time Decision	Yes	No	Limited
Updates Knowledge	Yes	No	No
Dynamic Environment	Excellent	Poor	Moderate
Suitable for Mars Rover	Yes	No	Limited

Online Search is specifically designed for environments where complete information is unavailable.

(v) Justify the most suitable approach considering uncertainty, adaptability, and safety.

The **Online Search Agent** is the most suitable approach.

Justification

- Operates without a complete map.
- Learns continuously during exploration.
- Updates decisions based on sensor data.
- Avoids newly discovered obstacles.
- Supports autonomous operation.
- Ensures safer navigation.

- Adapts to uncertain and changing environments.

These characteristics make Online Search the preferred solution for autonomous Mars exploration.

Algorithm

Algorithm: Online Search for Mars Rover Navigation

1. Start at the landing location.
2. Sense nearby terrain using onboard sensors.
3. Update the internal map with newly discovered information.
4. Check for hazards such as rocks and craters.
5. Select the safest neighboring location.
6. Move to the selected location.
7. Repeat sensing and map updating.
8. Collect scientific samples upon reaching the target.
9. Continue exploration until all objectives are completed.

Advantages

- Supports autonomous exploration.
- Learns continuously from the environment.
- Handles unknown terrain effectively.
- Avoids unexpected obstacles.
- Suitable for real-time decision making.
- Reduces mission risk.
- Improves navigation safety.

Applications

- Mars rover exploration.
- Space robotics.
- Autonomous drones.
- Search and rescue robots.
- Military surveillance robots.
- Underwater autonomous vehicles.
- Disaster response robots.

Result

The Online Search Agent successfully enables the Mars rover to explore unknown terrain, avoid hazards, and collect scientific samples while continuously updating its internal model. Unlike offline search methods, the online approach adapts to changing environmental conditions and makes intelligent real-time decisions based on newly acquired sensor information. Therefore, the Online Search Agent provides the most effective solution for autonomous Mars exploration by ensuring adaptability, safety, and efficient navigation under uncertain conditions.

QUESTION 4

University Examination Timetable Generation using Constraint Satisfaction Problem (CSP)

Problem Statement

A university is designing an Artificial Intelligence system to automatically generate examination timetables. The system must satisfy multiple constraints, including ensuring that no student has overlapping examinations, managing a limited number of examination halls, scheduling prerequisite subjects before dependent subjects, considering faculty availability, and providing special accommodations for eligible students. As the number of courses and students increases, the scheduling complexity grows significantly, making it an ideal Constraint Satisfaction Problem (CSP).

Problem Understanding

Preparing an examination timetable manually is a difficult and time-consuming task, especially in universities offering hundreds of courses. The timetable must satisfy several constraints simultaneously while ensuring fairness and efficient utilization of available resources.

Artificial Intelligence models this scheduling task as a **Constraint Satisfaction Problem (CSP)**. In a CSP, the objective is not to find the shortest path but to determine assignments that satisfy all specified constraints.

The timetable generation problem consists of:

- **Variables:** Subjects or examinations to be scheduled.
- **Domains:** Available dates, time slots, and examination halls.
- **Constraints:** Rules that every timetable must satisfy.
- **Goal:** Produce a conflict-free timetable while satisfying all academic and administrative requirements.

Since numerous combinations of schedules are possible, CSP techniques such as **Backtracking Search** and **Constraint Propagation (Forward Checking)** efficiently reduce the search space and generate valid timetables.

Method Selection

The university examination scheduling problem is solved using the **Constraint Satisfaction Problem (CSP)** approach.

A CSP consists of:

- Variables
- Domains
- Constraints

The search algorithm assigns values to variables one by one while checking whether each assignment satisfies all constraints. Whenever a conflict occurs, the algorithm backtracks and tries another assignment.

To improve efficiency, **Forward Checking** is used to eliminate invalid choices before they lead to conflicts.

Solution

Formulate the problem as a Constraint Satisfaction Problem (CSP).

Variables

Each examination subject is considered as a variable.

Example:

- Mathematics
- Physics
- Chemistry
- Computer Science
- English

Domains

Possible values for each variable include:

- Examination Date
- Time Slot
- Examination Hall

Example:

Dates:

Day 1

Day 2

Day 3

Time Slots:

Morning

Afternoon

Exam Halls:

Hall A

Hall B

Hall C

Constraints

The timetable must satisfy the following constraints:

- No student should have two examinations at the same time.
- One examination hall cannot conduct two examinations simultaneously.
- Faculty members should not have scheduling conflicts.
- Prerequisite subjects must be conducted before advanced subjects.
- Students requiring special accommodations must receive suitable examination arrangements.
- Every subject must be assigned exactly one valid time slot.

(i) Clearly define Variables, Domains, and Constraints with explanation.

Variables

The variables represent all examinations that need to be scheduled.

Example:

X1 = Mathematics

X2 = Physics

X3 = Programming

X4 = Artificial Intelligence

Domains

Each variable can be assigned any available examination slot.

Example:

Day 1 Morning

Day 1 Afternoon

Day 2 Morning

Day 2 Afternoon

Day 3 Morning

Day 3 Afternoon

Constraints

The timetable must satisfy:

- No overlapping examinations for any student.
- Hall capacity should not exceed available seats.
- Faculty availability must be respected.
- Laboratory examinations require laboratory rooms.
- Prerequisite subjects must occur before advanced subjects.
- Each examination must be assigned exactly one slot.

These constraints ensure a valid and conflict-free timetable.

(ii) Explain how Backtracking Search works in this scenario.

Backtracking Search assigns examination slots one subject at a time.

For example:

Mathematics → Day 1 Morning ✓

Physics → Day 1 Afternoon ✓

Programming → Day 2 Morning ✓

Artificial Intelligence → Day 2 Afternoon ✓

If assigning a subject violates any constraint, the algorithm removes the assignment and tries another available slot.

Example:

Programming → Day 1 Morning ✗

Conflict with Mathematics

↓

Backtrack

↓

Assign Programming → Day 2 Morning ✓

This process continues until all examinations are scheduled successfully.

(iii) Discuss how Constraint Propagation (Forward Checking) improves efficiency.

Forward Checking improves the search process by eliminating invalid assignments before they are selected.

For example:

Suppose

Mathematics → Day 1 Morning

Students enrolled in Mathematics cannot write another examination during the same time.

Forward Checking immediately removes

Day 1 Morning

from the domains of conflicting subjects.

Benefits include:

- Reduces unnecessary searching.
- Detects conflicts early.
- Minimizes backtracking.
- Speeds up timetable generation.
- Improves overall efficiency.

(iv) Analyze real-world challenges and justify the best strategy for scalability.

Real-World Challenges

- Large number of courses.
- Thousands of students.
- Limited examination halls.
- Faculty availability changes.
- Laboratory examinations require special facilities.
- Students with disabilities require additional arrangements.
- Last-minute examination changes.

Best Strategy

The most suitable approach is **Backtracking Search combined with Constraint Propagation (Forward Checking).**

Justification

- Generates conflict-free timetables.

- Handles multiple constraints simultaneously.
- Reduces computational complexity.
- Scales efficiently as the number of courses increases.
- Produces valid schedules within a reasonable time.

Algorithm

Algorithm: CSP using Backtracking and Forward Checking

1. Define all examination subjects as variables.
2. Define available dates, time slots, and halls as domains.
3. Specify all scheduling constraints.
4. Select an unassigned examination.
5. Assign a valid time slot.
6. Apply Forward Checking to remove conflicting choices.
7. If no valid assignment exists, backtrack.
8. Repeat until every examination is assigned.
9. Display the final examination timetable.

Advantages

- Produces conflict-free timetables.
- Handles multiple scheduling constraints.
- Reduces unnecessary computation.
- Efficient for large universities.
- Supports automatic timetable generation.
- Improves resource utilization.
- Easily adaptable to changing requirements.

Applications

- University examination scheduling.
- School timetable generation.
- Employee shift scheduling.
- Hospital duty scheduling.
- Airline crew scheduling.
- Sports tournament scheduling.
- Project resource allocation.

Result

The Constraint Satisfaction Problem (CSP) approach successfully generates a valid university examination timetable by satisfying all scheduling constraints. Backtracking Search systematically explores possible assignments, while Forward Checking improves efficiency by eliminating invalid choices before conflicts occur. Together, these techniques produce scalable, conflict-free timetables suitable for universities with a large number of courses, students, and resource constraints.

QUESTION 5

Strategic Game AI for Real-Time Decision Making using Minimax and Alpha-Beta Pruning

Problem Statement

A gaming company is developing an Artificial Intelligence opponent for a real-time strategy game. The AI must compete against human players by making intelligent decisions within strict time limits while evaluating complex game states containing multiple possible moves. Since the decision tree grows exponentially as the game progresses, exhaustive computation becomes impractical. Therefore, efficient game-search algorithms such as Minimax and Alpha-Beta Pruning are required to balance optimal decision-making with real-time performance.

Problem Understanding

Artificial Intelligence plays a vital role in modern strategy games by enabling computer-controlled opponents to make intelligent decisions. The AI must evaluate different possible moves, predict the opponent's responses, and choose the action that maximizes its chances of winning.

In a real-time strategy game, each move creates several new possibilities, causing the search tree to grow rapidly. Searching every possible move becomes computationally expensive and slows down decision-making.

To solve this problem, the AI uses **Minimax** to determine the optimal move and **Alpha-Beta Pruning** to eliminate unnecessary branches of the search tree, significantly improving efficiency without affecting the final decision.

The game search problem consists of:

- **Initial State:** Current game board or game state.
- **Goal State:** Reach a winning game position.
- **State Space:** All possible game configurations.
- **Actions:** Every legal move available to the player.
- **Utility Function:** Numerical score representing the quality of a game state.

Method Selection

The problem is solved using two Artificial Intelligence algorithms:

1. Minimax Algorithm

Minimax is an adversarial search algorithm used in two-player games.

It assumes:

- One player (**MAX**) attempts to maximize the score.
- The opponent (**MIN**) attempts to minimize the score.

The algorithm evaluates future moves and selects the action that leads to the best guaranteed outcome.

Advantages

- Produces optimal decisions.
- Suitable for competitive games.
- Predicts opponent's actions.
- Guarantees the best move when the entire game tree is explored.

Limitation

- Requires exploring a large number of game states.
- Computationally expensive for complex games.

2. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm.

It avoids evaluating branches that cannot influence the final decision.

By eliminating unnecessary nodes, Alpha-Beta Pruning:

- Reduces computation time.
- Explores fewer game states.
- Produces the same optimal decision as Minimax.

Solution

(i) Explain how the Minimax algorithm models this problem.

Minimax models the game as a search tree.

Each level of the tree alternates between:

- **MAX Player (AI)**
- **MIN Player (Opponent)**

Example:

MAX

/ \



The MIN player chooses the smaller value from each branch:

MIN Left = 3

MIN Right = 2

The MAX player then chooses the larger value:

MAX = max(3,2)

Best Move = 3

Thus, the AI selects the move that guarantees the highest possible outcome, assuming the opponent always plays optimally.

(ii) Analyze the computational challenges of large game trees.

As the number of possible moves increases, the game tree grows exponentially.

For example:

- Level 1 → 5 moves
- Level 2 → 25 states
- Level 3 → 125 states
- Level 4 → 625 states

This exponential growth creates several challenges:

- High computation time.
- Increased memory usage.
- Slow response in real-time games.
- Difficulty exploring every possible move before the time limit expires.

Therefore, optimizing the search process becomes essential.

(iii) Explain how Alpha-Beta Pruning improves efficiency with detailed reasoning.

Alpha-Beta Pruning reduces the number of nodes evaluated by eliminating branches that cannot affect the final decision.

Two values are maintained during the search:

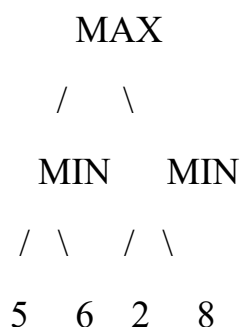
- **Alpha (α):** Best value found so far for the MAX player.
- **Beta (β):** Best value found so far for the MIN player.

If at any point:

$$\text{Alpha} \geq \text{Beta}$$

the remaining branches are ignored because they cannot improve the final result.

Example



After evaluating the left branch:

$$\text{Alpha} = 5$$

While evaluating the right branch:

$$\text{Beta} = 2$$

Since:

$$\text{Alpha } (5) \geq \text{Beta } (2)$$

the remaining branch containing **8** is pruned and not evaluated.

This significantly reduces the number of explored nodes while still producing the same optimal move.

(iv) Design an evaluation function and justify the factors considered.

An evaluation function estimates the quality of a game state when the search cannot continue to the end of the game.

Example Evaluation Function

Evaluation Score =

$(10 \times \text{Number of AI Units})$

$+ (5 \times \text{Resources Collected})$

$+ (20 \times \text{Territory Controlled})$

$- (8 \times \text{Enemy Strength})$

$- (5 \times \text{Damage Received})$

Factors Considered

- Number of surviving AI units.
- Resources available.
- Territory captured.
- Enemy strength.
- Health of important units.
- Defensive position.
- Probability of winning.

A higher evaluation score indicates a better game position for the AI.

(v) Recommend a strategy for balancing optimality and real-time performance.

The most suitable strategy is:

Minimax with Alpha-Beta Pruning

Justification

- Produces optimal game decisions.
- Eliminates unnecessary search.
- Reduces computation time significantly.
- Handles large game trees efficiently.
- Suitable for strict real-time constraints.
- Maintains the same decision quality as Minimax.

For highly complex games such as Chess, Checkers, or Real-Time Strategy (RTS) games, Alpha-Beta Pruning allows the AI to search deeper within the available time, improving overall gameplay performance.

Algorithm

Algorithm: Minimax with Alpha-Beta Pruning

1. Represent the current game state as the root node.
2. Generate all possible legal moves.
3. Apply the Minimax algorithm to evaluate future game states.
4. Maintain Alpha and Beta values during the search.
5. Prune branches whenever $\text{Alpha} \geq \text{Beta}$.
6. Continue until the specified search depth or terminal state is reached.
7. Select the move with the highest evaluation score.
8. Execute the selected move.

Advantages

- Produces optimal game decisions.
- Predicts opponent's moves.
- Reduces unnecessary computation.
- Improves search efficiency.
- Supports real-time decision making.
- Explores deeper game trees.
- Maintains optimality while reducing execution time.

Applications

- Chess AI.
- Checkers AI.
- Tic-Tac-Toe.
- Real-Time Strategy (RTS) games.
- Turn-based strategy games.
- Board game AI.
- Game-playing agents in Artificial Intelligence.

Result

The Minimax algorithm successfully models strategic decision-making by evaluating future game states and selecting the optimal move against an intelligent opponent. However, exploring every possible game state becomes computationally expensive as the game tree grows. Alpha-Beta Pruning overcomes this limitation by eliminating branches that cannot influence the final decision, thereby reducing computation time while maintaining the same optimal result. Consequently, **Minimax with Alpha-Beta Pruning** is the most suitable approach for developing efficient real-time game AI, providing an effective balance between optimal decision-making and computational performance.

